संथाली और भीली मशीनी अनुवाद के लिए इंडिकट्रांस२ के साथ शब्दावली विस्तार

Vocabulary Extension for Santali and Bhili Machine Translation with IndicTrans2

एक प्रतिवेदन

A Report

आवश्यकताओं की आंशिक पूर्ति हेतु प्रस्तुत

Submitted in Partial Fulfillment of the Requirements

प्रौद्योगिकी स्नातक
डिग्री हेतु

for the Degree of

Bachelor of Technology

संगणक विज्ञान एवं अभियांत्रिकी में

in Computer Science & Engineering

द्वारा

By

प्रतीक [२०२३३२१८] Prateek [20233218]

रोशना राजेश [२०२३३५५१] Roshna Rajesh [20233551]

रुचिर जोशी [२०२३३२३६] Ruchir Joshi [20233236]

संजीवनी जायसवाल [२०२३३२५२] Sanjeevani Jaiswal [20233252]

पर्यवेक्षक: Dr.Divyansh Thakur

Supervisor: Dr.Divyansh Thakur



संगणक विज्ञान एवं अभियांत्रिकी विभाग

COMPUTER SCIENCE AND ENGINEERING DEPARTMENT

मोतीलाल नेहरू राष्ट्रीय प्रौद्योगिकी संस्थान इलाहाबाद, प्रयागराज–२११००४, भारत

MOTILAL NEHRU NATIONAL INSTITUTE OF TECHNOLOGY ALLAHABAD, PRAYAGRAJ-211004, INDIA

नवम्बर, २०२६

November, 2026

UNDERTAKING

We declare that the work presented in this report titled "*Vocabulary Extension for Santali and Bhili Machine Translation with IndicTrans2*", submitted to the Computer Science and Engineering Department, Motilal Nehru National Institute of Technology Allahabad, Prayagraj, for the award of the ***Bachelor of Technology*** degree in ***Computer Science & Engineering***, is our original work. We have not plagiarized or submitted the same work for the award of any other degree. In case this undertaking is found incorrect, we accept that our degrees may be unconditionally withdrawn.

November, 2026
Allahabad

(Prateek [20233218],
Roshna Rajesh [20233551],
Ruchir Joshi [20233236],
Sanjeevani Jaiswal [20233252])

CERTIFICATE

Certified that the work contained in the report titled "*Vocabulary Extension for Santali and Bhili Machine Translation with IndicTrans2*", by *Prateek, Roshna Rajesh, Ruchir Joshi and Sanjeevani Jaiswal*, has been carried out under my supervision and that this work has not been submitted elsewhere for a degree.

November, 2026

(Dr.Divyansh Thakur)
Computer Science and Engineering Dept.
M.N.N.I.T, Allahabad

Preface

This thesis started as a plan to study four tribal languages of India (Bhili, Gondi, Mundari and Santali) and ask whether the tokenizer of a strong multilingual translation model holds them back. The experiments narrowed the plan to one controlled question: if the decoder is taught a few hundred whole target-language words as single tokens, does translation improve? Two languages were studied. Santali (English→Santali, Ol Chiki script) is the language IndicTrans2 officially lists. Bhili (Hindi→Bhili, Devanagari script) is not listed, so every Bhili run borrows the Marathi tag.

For Santali the honest answer is "a little". Because the effect is small, the project became as much about measurement as about modelling: every system of both languages uses one training procedure, the Santali baseline and the main Santali extension were each run with two seeds, and every difference carries a paired bootstrap interval. Bhili was run with the same recipe against a baseline trained in exactly the same way. Its gain is smaller still (at most 0.55 BLEU and 0.48 chrF) and appears only with the larger word lists, so we state which conclusions rest on Santali alone.

All experiments ran on a single free Kaggle T4 GPU. Things one would normally want (more seeds, a second Bhili baseline seed, a reimplementation of the shared-task pipeline) did not fit that budget; the limitations are listed in Chapter 7.

Acknowledgements

We would like to express our sincere gratitude to our supervisor, Dr. Divyansh Thakur, for his guidance and encouragement throughout this project. He steered us towards tribal languages, supported our decision to narrow the work down to Santali and Bhili.

We thank the Department of Computer Science and Engineering and Motilal Nehru National Institute of Technology Allahabad for the academic environment and the opportunity to carry out this mini-project.

We are grateful to the Ministry of Tribal Affairs, Government of India, for the *Adi Vaani* portal (`adivaani.tribal.gov.in`), which first showed us how much work is being done on tribal languages and pointed us to the scripts and resources we needed. We thank the organisers of the MMLoSo 2025 shared task for releasing the Santali and Bhili parallel data, and the participating teams, especially the authors of the winning challenge-track system, whose published work shaped our choice of backbone and of the proxy language tag for Bhili.

We thank the AI4Bharat team for releasing IndicTrans2, its tokenizer and IndicTransToolkit, and the IN22 benchmark openly, and the open-source communities behind Hugging Face Transformers, PEFT, SentencePiece and sacreBLEU, on which every experiment rests. We thank Kaggle for the free GPU hours behind every run.

Finally, we thank the Santali and Bhili speaking communities, whose languages this work hopes to serve, and our friends and families for their patience and support through a semester of us muttering about tokens.

Contents

Preface **iv**

Acknowledgements **v**

1 Introduction **1**
1.1 Motivation . . . 1
1.2 Background . . . 2
1.3 Problem Statement . . . 3
1.4 Research Questions . . . 4
1.5 Contributions . . . 4
1.6 Scope . . . 5
1.7 Organisation . . . 5

2 Related Work **6**
2.1 Low-Resource NMT, IndicTrans2 and the Data . . . 6
2.2 Tokenization, Vocabulary Extension and LoRA . . . 6
2.3 Evaluation and Statistical Reliability . . . 7
2.4 Closest Work and Gap . . . 7

3 Methodology **9**
3.1 Overview . . . 9
3.2 Candidate Word Selection . . . 9
3.3 Label Encoding . . . 11
3.4 Modifying the Model . . . 11
3.5 Initialising the New Rows . . . 12
3.6 Integrity Checks Before Training . . . 12
3.7 Training Procedure . . . 13
3.8 Decoding Added Tokens . . . 14
3.9 Experimental Design . . . 14

4 Experimental Setup **17**
4.1 Data . . . 17
4.2 Model, Software and Hardware . . . 18
4.3 Training Configuration . . . 19
4.4 Evaluation Protocol . . . 19
4.5 Tokenizer Audit . . . 21
4.5.1 Vocabularies and checkpoints . . . 21
4.5.2 Fertility and word splitting . . . 21
4.5.3 Unknown tokens, round trips and digits . . . 22
4.5.4 Sanity checks before any training . . . 22

5 Results **24**
5.1 Main Results . . . 24
5.2 Paired Bootstrap . . . 24
5.3 Effect of the Number of Added Words . . . 26
5.4 Mean Versus Random Initialisation . . . 27
5.5 Seed Variation . . . 28
5.6 Where the Gain Comes From . . . 28
5.7 Training Behaviour . . . 30
5.8 Baseline Comparability . . . 31
5.9 External Evaluation on IN22-Gen . . . 32

6 Discussion **34**
6.1 Santali: the Controlled Result . . . 34
6.2 Bhili: a Replication With a Matched Baseline . . . 34
6.3 The Methodological Lesson . . . 35
6.4 Claims the Evidence Supports and Does Not Support . . . 36

7 Conclusion and Future Work **37**
7.1 Conclusion . . . 37
7.2 Limitations . . . 38
7.3 Future Work . . . 39

References **41**

Chapter 1

Introduction

Machine translation has improved enormously for languages with a lot of text, and much less for languages without it. Santali is a Munda language written in its own Ol Chiki script and spoken by about 7.4 million people [9]. It is one of the 22 scheduled languages of India, and IndicTrans2 [3] brought it into a single open translation model. What that model does with Santali internally depends partly on its tokenizer: the target side uses a SentencePiece [7] vocabulary of 128,000 pieces shared across many scripts. This thesis tests one remedy: give the model whole-word tokens for the most frequent target-language words that the tokenizer splits. It does so for two languages. Santali is the main study (matched baselines, two baseline seeds, an external benchmark). Bhili, an Indo-Aryan language of the Bhil communities written in Devanagari in the MMLoSo 2025 data, is the replication (matched baseline, one baseline seed, no external benchmark). IndicTrans2 has no Bhili tag, so every Bhili run conditions on the Marathi tag `mar_Deva` as an unverified proxy.

1.1 Motivation

Tokenization sets how much work the decoder has to do. A word spelled with three pieces needs three correct decisions in a row; a single-token word needs one (Figure 1). Heavier fragmentation tends to go with worse modelling and higher cost [1, 11, 15], and recent work on tribal-language translation treats the tokenizer's representation of a language as a first-order factor [18].

We measured the starting point rather than assuming it (Section 4.5). On 2,000 training sentences the 128k vocabulary gives 1.442 pieces per Santali word against 2.029 for the older 32k vocabulary (split words fall from 99.6% to 27.2%), and 1.448 against 4.957 per Bhili word (98.3% to 32.4%). The large vocabulary already does much of the job; the question is whether the remaining quarter to third of

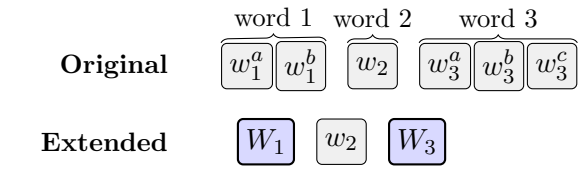


Figure 1: Schematic of the idea. A frequent word that the original tokenizer splits (grey) becomes one added token (blue); words that are already one piece are untouched. Six decisions become three.

split words is worth fixing, and whether fixing it helps translation or only shortens sequences. Extension is cheap: it needs no new pretraining and only a few hundred extra embedding rows, which fit next to a LoRA adapter on a small GPU.

1.2 Background

Why Santali and Bhili

The MMLoSo 2025 corpus has 20,000 sentence pairs each for Bhili, Mundari, Gondi and Santali. Of these four, Santali is the language with an official IndicTrans2 tag (`sat_Olck`), so it can be adapted with no workaround, and we started from it. The tokenizer audit then supported the choice: in the notebook outputs Santali has the highest unknown-token rate of all four languages (on 100 validation sentences each: Santali 1.98%, Mundari 0.75%, Gondi 0.17%, Bhili 0.06% of target tokens; Section 4.5), which makes it a demanding test of whether whole-word tokens help. Bhili has no IndicTrans2 tag. The winning challenge-track paper [14] maps it to the Marathi tag `mar_Deva`, and this inspired us to use the same proxy; that paper describes such a proxy for Bhili only, not for Gondi or Mundari, so Bhili became our second language. The audit also shows that Bhili is much more fragmented than Santali under the older 32k vocabulary (4.957 pieces per word against 2.029), and under the 128k vocabulary a third of its words are still split (32.4% against 27.2%), so it is a natural case for the vocabulary question.

Santali is a Munda language of the Austroasiatic family, written in the Ol Chiki script devised by Raghunath Murmu in 1925. It was added to the Eighth Schedule by the 92nd Constitutional Amendment (2003), the 2011 Census recorded 7,368,192 speakers [9], making it the most widely spoken tribal language in India, and IndicTrans2 officially supports it. The Santali pair is English→Santali in Ol Chiki. Bhili is written in Devanagari in this dataset (pair Hindi→Bhili) and is not listed by IndicTrans2, so it stands for the harder case of a language with no official tag.

National context and social causes

The Viksit Bharat 2047 vision is measured not only by economic output but by whether public services, the digital economy and knowledge reach every citizen in the language spoken at home; information in English or Hindi is out of easy reach for a first-language Santali or Bhili speaker. The National Education Policy 2020 stresses mother-tongue instruction and the BHASHINI mission aims to put digital services in Indian languages, both of which need translation that works beyond the largest few languages. Better Santali and Bhili translation serves education (textbooks and study material), health and public information (advisories, vaccination, disaster warnings), access to government schemes and rights (with human review), digital inclusion, and cultural and linguistic heritage. These uses need translation that is accurate and cheap enough to deploy; large multilingual models are costly to fine-tune and long subword sequences make generation slower and less reliable, so a method that makes a language more compact for the model at little cost supports them directly. Output should be reviewed by speakers where errors carry real consequences.

The technical problem and the backbone

A pretrained model has a fixed subword vocabulary; when a language is poorly represented its words split into many small pieces, so the model generates longer sequences from less informative units. We add frequent whole target words to the decoder vocabulary and start the new embeddings from the pieces they replace. Chinese-LLaMA and multilingual-BERT vocabulary expansion [19] are precedents, but controlled evidence for low-resource translation into a new script is limited. Shared-task teams reported that NLLB-200 fragments Ol Chiki (about 3.0 pieces per word against 1.44 for IndicTrans2) and the winning system switched to IndicTrans2 1B [14, 17], so we use `indictrans2-indic-indic-1B` with LoRA. The MMLoSo system papers we read used adapters, retrieval, reranking or frozen encoders; none extended the vocabulary, which is the gap this work fills.

1.3 Problem Statement

Given IndicTrans2 1B (`ai4bharat/indictrans2-indic-indic-1B`) and a parallel corpus of about 18,000 training sentences per language (English–Santali, Hindi–Bhili), determine whether extending the decoder vocabulary with frequent whole target words improves translation under LoRA adaptation, compared with a baseline trained and evaluated in the same way but keeping the original vocabulary. For Bhili the target has no tag of its own, so every Bhili run conditions on a proxy tag.

1.4 Research Questions

Table 1 lists the six questions with short answers. The first five are asked of both languages where the controls allow (Santali has all of them, Bhili lacks an external benchmark and a second baseline seed); the last asks whether the Bhili result replicates the Santali one.

Question	Short answer
RQ1 Size: how does quality change with the number of added words (Santali 95, 445, 1,220; Bhili 166, 510, 1,173, 2,729)?	Santali: no effect at 95, a modest gain at 445, no further gain at 1,220. Bhili: no measurable effect at 166 and 510, a small gain from 1,173 words that does not grow at 2,729.
RQ2 Initialisation: does mean initialisation beat random?	Unresolved in both languages (differences inside noise); in Bhili the random run is not distinguishable from the baseline.
RQ3 Source of gain: is the gain concentrated on sentences with added words?	Partly in Santali (control words improve too). In Bhili the chrF gain sits mostly in sentences whose output or reference has an added word; BLEU is less clear.
RQ4 Reliability: how large is seed noise?	About 0.4 BLEU and 0.4 chrF in Santali (baseline seed 43 against 42). Bhili extension seeds differ by about 0.1 BLEU and chrF, but the Bhili baseline has one run, so its seed noise is unknown.
RQ5 Transfer: does the gain survive on IN22-Gen?	Santali: yes on chrF++ (+0.56 to +0.85), weak on BLEU. Bhili: no external benchmark.
RQ6 Replication: does the Bhili result, against a matched baseline, show the Santali pattern?	Partly. The gain is smaller (at best +0.55 BLEU, +0.48 chrF), appears only from 1,173 words and is about the size of Santali's seed spread.

Table 1: Research questions and short answers.

1.5 Contributions

- A reproducible implementation of whole-word vocabulary extension for an encoder–decoder model whose decoder embedding and output layer are separate matrices, with integrity checks (Chapter 3).
- A controlled Santali dose–response study with matched training, two baseline seeds, an initialisation ablation, paired bootstrap intervals and an IN22-Gen evaluation, plus a word-level analysis against a control set.
- A replication for Hindi→Bhili through a proxy tag (four sizes, a second seed, a random-initialisation run) against a baseline trained with the same loop, analysed in parallel with Santali.

- A tokenizer audit (fertility, unknown tokens, round trips, Ol Chiki digits) for both languages.

1.6 Scope

Two language pairs, one model, one question. Gondi and Mundari were prepared by the same pipeline but are not used. Bhili has one baseline seed and no external benchmark, so it is a replication with weaker control than Santali. A reimplementation of the published shared-task pipeline was dropped because full fine-tuning of a one-billion-parameter model does not fit a 15 GB T4; no claim is made about leaderboard scores.

1.7 Organisation

Chapter 2 reviews related work, Chapter 3 the method, Chapter 4 the data, training, evaluation and tokenizer audit, Chapter 5 the results for both languages, Chapter 6 the discussion, and Chapter 7 conclusions, limitations and future work.

Chapter 2

Related Work

2.1 Low-Resource NMT, IndicTrans2 and the Data

Transformer encoder–decoder models degrade quickly when parallel data is scarce, and the usual response is to adapt a large multilingual model; NLLB [8] extended coverage to hundreds of languages, yet the tail still scores lowest (and fragments Ol Chiki into about 3.0 pieces per word). IndicTrans2 [3] covers all 22 scheduled Indian languages and ships the IN22 benchmark. We use the 1B Indic–Indic checkpoint, following the winning MMLoSo 2025 challenge-track system [14]. English enters through the tag `eng_Latn` although this checkpoint was not trained with English as a source, one reason the unadapted model scores low (Section 5.9); the Santali tag is `sat_Olck`. Fine-tuning data comes from MMLoSo 2025 [17], which also supplies Hindi–Bhili pairs. Bhili has no IndicTrans2 tag, and the winning system maps it to the Marathi tag `mar_Deva`, a surrogate sharing the Devanagari script [14]; we follow that convention. Because the preprocessing code accepts any language code (Section 4.5), every Bhili result concerns the proxy configuration, not native support. One participant study reports 7.3 BLEU and 40.3 chrF++ on IN22-Gen for English→Santali with extra public data [16]; this is orientation, not a target.

2.2 Tokenization, Vocabulary Extension and LoRA

Subword vocabularies shared across many languages favour the ones with the most text; tokenizer quality correlates with downstream performance [15], and tokens needed per sentence vary widely across languages [1, 11]. Adding tokens to a pre-trained model is not new: Wang et al. [19] added subwords for poorly covered languages. The key practical question is how to initialise new rows. Random initialisation discards what the model knows; averaging a word's constituent piece embeddings [4] uses it, and is the simplest member of the family of schemes such as

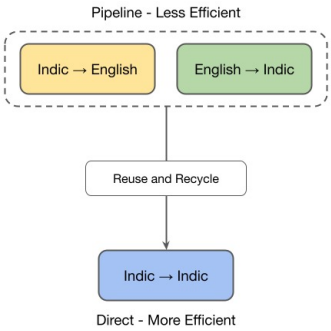


Figure 2: Why IndicTrans2 has a direct Indic–Indic model. Figure from the IndicTrans2 paper [3], reproduced with attribution.

WECHSEL. Most work concerns encoder-only or decoder-only models; for translation the output side matters most, so we extend both the decoder embedding and the output layer, which in IndicTrans2 are separate matrices (Section 3.4). LoRA [5] freezes pretrained weights and trains low-rank updates; it fits a 1B model on one T4 and is used for every system here (0.29% of the 1.21B parameters trainable).

2.3 Evaluation and Statistical Reliability

We report corpus BLEU [10], chrF [12] and TER with sacreBLEU [13] (chrF++ on the external benchmark); for a morphologically rich language chrF is generally more informative than BLEU. Differences are tested with the paired bootstrap [6]. Optimiser and data order alone can change fine-tuning results by amounts rivalling published improvements [2], and a bootstrap over test sentences says nothing about training randomness, so we run a second seed for the Santali baseline and for the main extension of both languages.

2.4 Closest Work and Gap

Singh et al. [18] study seven tribal languages (including Bhili and Santali) with IndicTrans2, NLLB-600M and ByT5, comparing native, Devanagari- and Latin-unified scripts. Script conversion is model-dependent, the byte-level model is much less sensitive, and reduced fragmentation does not reliably improve translation (Kui under IndicTrans2: fertility 6.05 to 2.39, chrF++ change only -1.8). Santali is a boundary case with the highest tokenization premium. Their experiments change script and leave each vocabulary fixed; we keep the script and change the vocabulary,

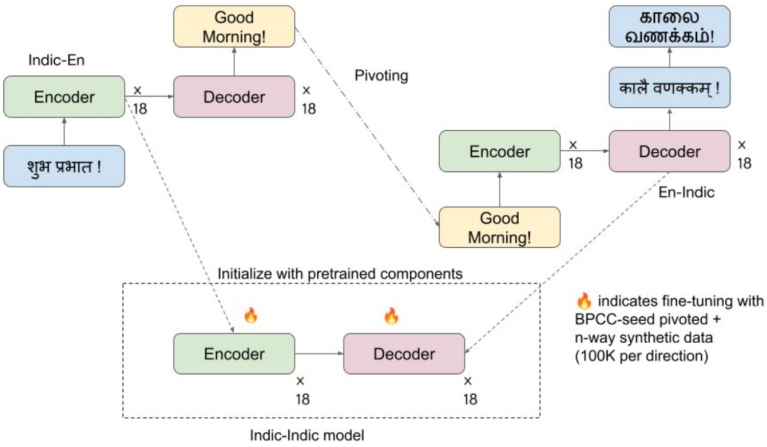


Figure 3: How the Indic–Indic model is built from the Indic–English encoder and English–Indic decoder. Figure from [3], reproduced with attribution.

for two directions in depth. Among the MMLoSo 2025 system papers we read, none extended the model's vocabulary, and vocabulary-expansion work rarely compares against a baseline trained under an identical procedure or reports seed variation. This thesis supplies a matched baseline for both languages and seed variation for Santali; Bhili adds a second language in which the matched gain is smaller and appears only with longer word lists.

Chapter 3

Methodology

3.1 Overview

The method is a small modification of the standard LoRA fine-tuning recipe, identical for both languages:

1. Freeze the data splits once (Section 4.1) and audit the original tokenizer on the target text.
2. Select candidate words from the *training* split only, by a frequency threshold.
3. Encode training labels so a candidate becomes one new token ID wherever it appears as a whole word.
4. Add trainable embedding and output rows for the new IDs (mean or random initialisation) and attach LoRA.
5. Train with a fixed procedure, keeping the best checkpoint by validation loss.
6. Decode generated IDs with a decoder that handles the new tokens, and score under one fixed pipeline.

The source-side tokenizer is never changed. Only the target labels and the decoder's two vocabulary-sized matrices differ from the baseline. Only the data files, language tags, thresholds and beam width (4 for Santali, 5 for Bhili) differ between languages. Figure 4 shows the pipeline.

3.2 Candidate Word Selection

A candidate is a whole whitespace-delimited word from the training targets that occurs at least `MIN_FREQ` times, has no punctuation or symbols, and is split into

more than one piece by the original SentencePiece model. Candidates are sorted by frequency, then pieces saved, then spelling, and new IDs are assigned from 122,672 upward. Table 2 lists the nested lists for both languages (Santali has 41,757 unique training words, Bhili 62,328). Four lists per language were trained; the shortest and longest Santali lists and the shortest Bhili list were measured only.

Language	Min. freq.	Words added	Token reduction (%)	Trained
Santali	100	21	0.74	no
	50	95	1.89	yes
	20	445	4.73	yes
	10	1,220	7.36	yes
	5	3,020	10.17	no
Bhili	100	60	–	no
	50	166	4.35	yes
	20	510	6.61	yes
	10	1,173	8.79	yes
	5	2,729	11.26	yes

Table 2: Candidate lists by frequency threshold. Santali reductions are measured on held-out validation text, Bhili reductions on 2,000 training sentences (31.233 pieces per sentence originally), so the two columns are not strictly comparable. Lists are nested.

On the training sample, Santali pieces per word fall from 1.4421 to 1.4137, 1.3719 and 1.3310 with 95, 445 and 1,220 words; Bhili pieces per sentence fall from 31.233 to 29.875, 29.170, 28.488 and 27.717 with 166, 510, 1,173 and 2,729 words. For Santali, the 445-word saving on validation text (4.73%) tracks the 4.87% on the training sample, so the list is not overfitted to the training text. The core of the selection, abridged, is:

```
word_counts = Counter()
for t in train_texts:
    word_counts.update(t.split())
rows = []
for w, f in word_counts.items():
    if f < MIN_FREQ or only_punct(w):
        continue
    pieces = sp.encode(w, out_type=str)
    if len(pieces) > 1:          # already-single pieces gain nothing
        rows.append((w, f, len(pieces)))
```

3.3 Label Encoding

A candidate is replaced by its new ID only when it forms a whole word: it starts the text or follows whitespace, and ends the text or is followed by whitespace, common punctuation or the danda. Other text goes through the original tokenizer unchanged. Two properties are asserted before training: a sentence with no candidate encodes *exactly* as the original tokenizer would, and decoding an extended encoding gives the same string as decoding the original.

```
def ext_encode(text, add_eos=False):
    ids, cursor = [], 0
    for m in added_pattern.finditer(text):
        ids += encode_original(text[cursor:m.start()])
        ids.append(candidate_to_id[m.group(1)])
        cursor = m.end()
    ids += encode_original(text[cursor:])
    if add_eos and (not ids or ids[-1] != EOS_ID):
        ids.append(EOS_ID)
    return ids
```

3.4 Modifying the Model

IndicTrans2's configuration reports tied embeddings, but the checkpoint has separate decoder-embedding and output-projection matrices of 122,672 rows each (the encoder has 122,706). Extending only one would leave the model unable to read or produce the new tokens, so both are extended (Figure 5). Each matrix becomes a module holding the original rows frozen and N new rows kept in fp32 and trainable, concatenated in the forward pass; fp32 avoids the instability of updating tiny numbers in half precision.

```
class ExtEmbedding(nn.Module):
    def __init__(self, original_weight, new_weight, padding_idx=None):
        super().__init__()
        self.padding_idx = padding_idx
        self.original_weight = nn.Parameter(
            original_weight.detach().clone(), requires_grad=False)
        self.new_weight = nn.Parameter(
            new_weight.detach().clone().float(), requires_grad=True)
    def forward(self, input_ids):
        w = torch.cat([self.original_weight,
                       self.new_weight.to(self.original_weight.dtype)])
        return F.embedding(input_ids, w, padding_idx=self.padding_idx)
```

The output layer has the same structure with a linear forward pass. Table 3 gives the trainable parameters; each added word adds 2,048 (two rows of 1,024) to the 3,538,944 LoRA parameters, and the notebooks assert these counts.

Language	Words added	New-row parameters	Total trainable	Output layer size
Santali	95	194,560	3,733,504	122,767
Santali	445	911,360	4,450,304	123,117
Santali	1,220	2,498,560	6,037,504	123,892
Bhili	166	339,968	3,878,912	122,838
Bhili	510	1,044,480	4,583,424	123,182
Bhili	1,173	2,402,304	5,941,248	123,845
Bhili	2,729	5,588,992	9,127,936	125,401

Table 3: Trainable parameters per setting.

3.5 Initialising the New Rows

Mean (compositional): a new row is the average of the rows of the SentencePiece pieces making up the word, in both the embedding and output matrix,

$$e_w = \frac{1}{|P(w)|} \sum_{p \in P(w)} e_p, \tag{1}$$

using only pieces present in the vocabulary (the global mean row if none is). **Random:** rows are drawn from a normal distribution matching the original matrix's mean and standard deviation, with a seeded generator. With E, H the frozen original matrices and E', H' the new rows, the model uses $\tilde{E} = [E; E']$ and $\tilde{H} = [H; H']$.

The step-0 validation loss with mean initialisation is finite and sensible for Santali (3.0125, 3.2583 and 3.5038 for 95, 445 and 1,220 words, rising slightly as more tokens are swapped in). For Bhili it is much higher (8.78 to 8.92), but the untuned baseline model gives 8.73 on the full Bhili validation set (and a similar loss on a single batch, 9.41 and 8.96 in two checks), so the high values reflect the unsupported language and proxy tag and not an initialisation fault.

3.6 Integrity Checks Before Training

Santali. With 95 words added, the extension-aware encoder reproduced the original IDs on all 672 validation sentences containing no candidate (0 mismatches), and isolated round trips passed for 20 words. In the 1,220-word run, 3 added words contain a piece missing from the target vocabulary: the baseline turns such pieces

into <unk>, the extension keeps the word whole, which can add a small unrelated gain (1,217 words averaged all their pieces; 3 used the global mean). Decoding 600 references through the extension-aware decoder matched the original decoder, apart from those words.

Bhili. The encoder reproduced the original IDs with zero mismatches on candidate-free validation sentences, and no added Bhili word has a piece missing from the vocabulary (0 of 2,729). Decoder parity passed for the 166-word run (0 of 600 differences) but *did not fully pass* for the 510-, 1,173- and 2,729-word runs and the seed-43 run: each reports 2 whitespace-only differences around parentheses, and their own output says training should wait for review, yet training ran. Two of 600 references are affected, so the effect is probably small, but it was not measured (Section 7.2).

3.7 Training Procedure

Every system of both languages, the baselines included, uses one recipe: LoRA rank 16 on q_proj and v_proj, AdamW, linear schedule with warmup, fp16 autocast with a gradient scaler, gradient clipping and early stopping on validation loss (Table 6).

```
lora_cfg = LoraConfig(r=16, lora_alpha=32,
                      target_modules=["q_proj", "v_proj"],
                      lora_dropout=0.05, bias="none",
                      task_type=TaskType.SEQ_2_SEQ_LM)
lora_model = get_peft_model(model, lora_cfg)
```

The loop unscales gradients once per accumulation group, skips the update if any gradient is non-finite, and clips the global norm in float64 (abridged):

```
scaler.unscale_(optimizer)
gn = grad_global_norm_float64(trainable_params)   # None if non-finite
if gn is not None:
    coef = min(1.0, MAX_GRAD_NORM / (gn + 1e-12))
    for p in trainable_params:
        if p.grad is not None: p.grad.mul_(coef)
    scaler.step(optimizer)
scaler.update()
```

Data order uses a generator seeded with SEED + epoch, so it is reproducible and survives resuming interrupted Kaggle sessions, and a full checkpoint (weights, optimiser, scheduler, scaler, history, RNG state) is written every epoch.

The baselines use this same loop, data order and checkpoint logic; the only difference from an extension run is that the vocabulary and the two vocabulary-sized matrices are left unchanged, so every comparison is matched (Section 5.8).

3.8 Decoding Added Tokens

Generated IDs $\geq$ 122,672 are mapped back to words and the text around them is decoded normally. A naive join glues words to neighbouring text, so the decoder inserts a space before an added word (unless the next piece continues it) and none before closing punctuation. Both languages use this segment-wise decoder. The effect is not small: for a 95-word model, re-decoding the *same* token IDs with correct spacing changed 441 of 994 predictions in whitespace only and moved BLEU from 11.77 to 12.38 with chrF unchanged, and a continuation-aware decoder gave 12.40. Decoding details can matter by more than the effect under study.

3.9 Experimental Design

Table 4 lists every trained system. Santali comparisons: each extension against the baseline of the same seed; 445 words at two seeds for repeatability; mean against random initialisation; the three sizes against each other. Bhili comparisons: each extension against the single baseline; the four sizes; the second seed and the random-initialisation run at 2,729 words. A second Bhili baseline seed did not fit the compute budget.

Language	Arm	Words added	Init	Seed
Santali	Baseline	0	–	42
	Baseline (seed control)	0	–	43
	Extension, freq. 50	95	mean	42
	Extension, freq. 20	445	mean	42
	Extension, freq. 20, second seed	445	mean	43
	Extension, freq. 20, random init	445	random	42
	Extension, freq. 10	1,220	mean	42
Bhili	Baseline	0	–	42
	Extension, freq. 50	166	mean	42
	Extension, freq. 20	510	mean	42
	Extension, freq. 10	1,173	mean	42
	Extension, freq. 5	2,729	mean	42
	Extension, freq. 5, second seed	2,729	mean	43
	Extension, freq. 5, random init	2,729	random	42

Table 4: Trained systems. All use the frozen split of their language, the shared recipe and one evaluation script per language.

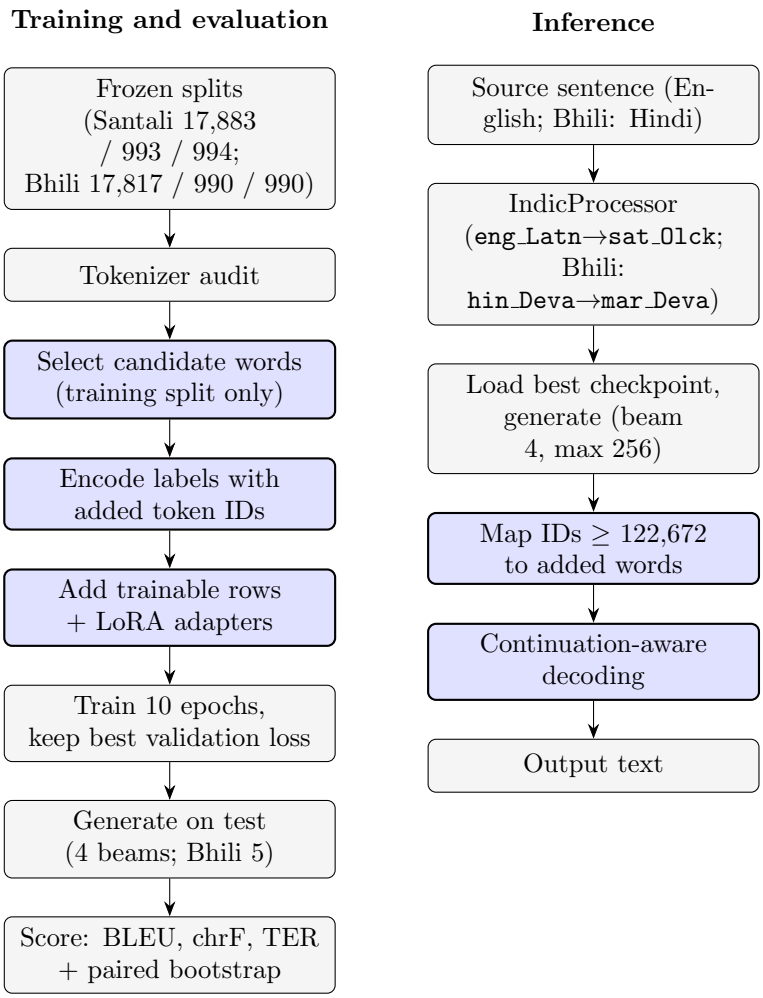


Figure 4: Training and evaluation pipeline (left) and inference path (right). Blue boxes differ from the baseline, which keeps the original vocabulary. Santali values are shown; Bhili values that differ are in parentheses.

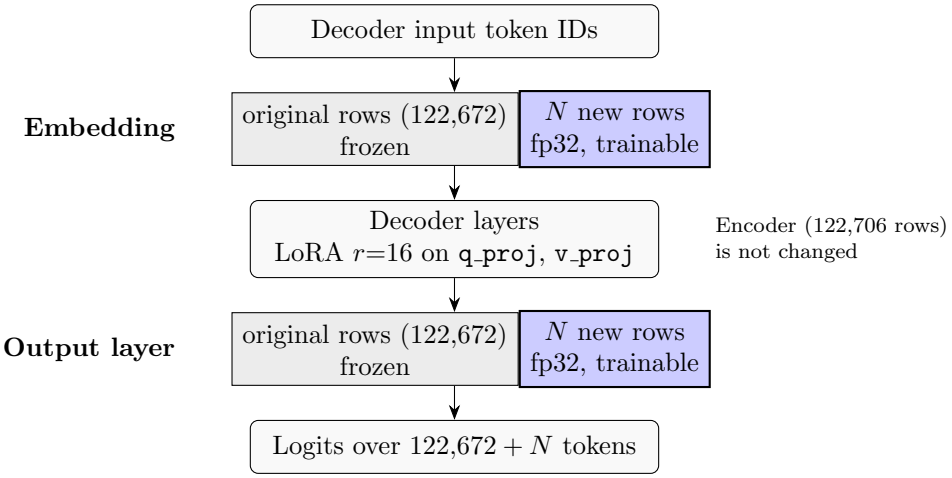


Figure 5: The modified decoder. Both vocabulary-sized matrices are the original frozen rows plus N new trainable rows (N from 95 to 2,729).

Chapter 4

Experimental Setup

4.1 Data

Both datasets come from the MMLoSo 2025 shared task [17] (20,000 rows each) and were cleaned by one shared notebook with one rule set: keep the two text columns, convert to strings and strip whitespace, drop missing or empty rows, drop duplicate pairs, reset the index, then split once with seed 42 into 90% training, 5% validation and 5% test. A post-cleaning check found no missing, empty, padded or non-string values or duplicate pairs, and every experiment notebook asserts zero pair overlap between splits and loads the same saved Parquet files. The competition's own test set was not available as a labelled public set, so every number comes from this local held-out split; it is an internal evaluation, not a leaderboard result. Table 5 gives the statistics.

Statistic	Santali	Bhili
Direction and target script	English→Santali, Ol Chiki	Hindi→Bhili, Devanagari
Raw rows	20,000	20,000
Cleaned rows	19,870 (130 removed)	19,802 (198 removed)
Train / validation / test as used	17,883 / 993 / 994	17,817 / 990 / 990
Rows whose source repeats	2,412 (about 12%)	472 (about 2.4%)
Sources with several translations	1,184	231
Rows whose target repeats	73	36
Source–target pairs shared between splits	0	0

Table 5: Data statistics for the two languages. The Bhili split sizes are those loaded by the experiment notebooks; the preprocessing notebook reports 17,821 / 990 / 991.

Santali

The English source side was produced by translating curated Hindi text with IndicTrans2 and lowercasing it, and the task data was already filtered to 6–80 words. Santali uses the officially supported pair `eng_Latn`→`sat_Olck` (source tag IDs 4 and 29,925). Caveats: (i) *possible source leakage*: splits are checked at pair level, so a repeated English source (2,412 rows; 1,184 sources with several Santali translations) may sit in both training and test with a different translation, which was not measured and may flatter all systems alike; (ii) *reference quality* was not audited; (iii) the saved preprocessing configuration names the Indic–English 1B checkpoint although every experiment used the Indic–Indic 1B checkpoint, which does not affect the splits.

Bhili

The source is Hindi and the target is Bhili, both in Devanagari. Bhili is not listed by IndicTrans2, so every run uses `hin_Deva` (ID 8) as source and the Marathi tag `mar_Deva` (ID 18) as a proxy target, chosen because Marathi shares the script, is geographically close, and was the choice of the rank-1 team [14]. The preprocessor accepts any code (`xyz_Test` and `bhili_Deva` were both accepted in a probe), so acceptance is no evidence of support. Caveats: (i) *split-size discrepancy*: the experiment notebooks load 17,817 training and 990 test rows whereas the preprocessing notebook reports 17,821 and 991, unexplained, so the exact dataset version should be documented; (ii) *reference quality*: in the printed rows the first training pair does not appear to translate its Hindi source, and another reference reads "Shanghai Cooperation Party" for the Hindi word for "organisation" (our reading of printed rows, not an audit); (iii) *possible source leakage* as for Santali, though Bhili is less exposed (472 repeated sources).

Gondi and Mundari were prepared by the same pipeline (20,000 rows each) but are not used.

4.2 Model, Software and Hardware

All runs fine-tune `ai4bharat/indictrans2-indic-indic-1B`, with source preprocessing by IndicProcessor, on a Kaggle Tesla T4 (about 14.6 GiB usable) with transformers 4.46.3, tokenizers 0.20.3, IndicTransToolkit 1.1.1, peft 0.20.0, torch 2.11.0+cu128, accelerate 1.14.0, sacrebleu 2.6.0 and Python 3.13.15 for both languages. The two Hugging Face packages are pinned so tokenizer behaviour is identical in every notebook.

4.3 Training Configuration

Setting	Value
LoRA	r=16, α=32, dropout 0.05, on `q_proj`, `v_proj`
Trainable parameters (baseline)	3,538,944 (0.29% of 1.21B)
Batch size / accumulation	4 / 4 (effective 16)
Optimiser	AdamW, lr 10^{-4}, weight decay 0.01
Schedule	3% warmup, then linear decay (Santali 11,180 steps, 335 warmup; Bhili 11,140 steps, 334 warmup)
Precision	fp16 autocast, gradient scaling, non-finite updates skipped
Gradient clipping	1.0 (global norm)
Epochs	10 planned, early stopping patience 2, min. delta 0.001
Model selection	best validation loss
Max. sequence length	256

Table 6: Training configuration shared by every arm of both languages, baselines included.

A Santali epoch takes about 15.5 to 18.7 minutes on the T4 and a Bhili epoch 14.0 minutes for the baseline and 14.9 to 16.3 minutes for the extensions. Every run finished all ten epochs, so early stopping never triggered, and only two to five optimiser updates per run were skipped for non-finite gradients (three for the Bhili baseline).

4.4 Evaluation Protocol

Both languages use the same procedure: IndicProcessor prompts, batch size 8, beam search, no sampling, the same script and decoder for every system, and sacreBLEU 2.6.0 defaults for corpus BLEU, chrF and TER (BLEU: mixed case, 13a tokenisation, exponential smoothing; chrF: character order 6, no word n-grams). Intervals are paired bootstrap over test sentences with 1,000 resamples; they cover test-sentence sampling only, not training randomness. The test set is used once per system. Table 7 gives the settings that differ.

Santali

Santali is scored on the 994 test sentences plus the first 300 validation sentences as a secondary check, and the trained systems are also run, without further training,

	Santali	Bhili
Test sentences	994	990
Beams / length limit	4 beams, `max_length` 256, early stopping	5 beams, `max_new_tokens` 256
Secondary check	first 300 validation sentences scored (BLEU, chrF)	300 validation sentences decoded but not scored
External benchmark	IN22-Gen (1,024 pairs), chrF++ added	none
Baseline	same loop as extensions, seeds 42 and 43	same loop as extensions, seed 42 only

Table 7: Evaluation settings per language. The two languages use different beam widths and data, so absolute scores are not comparable.

on IN22-Gen (Section 5.9). Because every baseline and extension share a procedure and the baseline has two seeds, a difference can be judged against both test-sentence noise and seed noise. The bootstrap, abridged, is:

```
def boot(a, b, B=1000, seed=0):
    rng = np.random.default_rng(seed); n = len(refs); db, dc = [], []
    for _ in range(B):
        idx = rng.integers(0, n, n); r = [refs[i] for i in idx]
        x = [hyps[a][i] for i in idx]; y = [hyps[b][i] for i in idx]
        db.append(bleu(y, r) - bleu(x, r))     # bleu/chrf = sacrebleu
        dc.append(chrf(y, r) - chrf(x, r))     # corpus scores
    return db, dc     # 2.5th, 97.5th percentiles = 95% interval
```

Bhili

Bhili is scored on the 990 test sentences. One comparison notebook loads the saved test predictions of the baseline and of all six extension runs, checks that sources, references and order are identical, and applies the same paired bootstrap (1,000 resamples, one fixed seed) to every run. No validation BLEU or chrF was reported and no external benchmark exists, so there is no secondary check of the kind used for Santali. With a single baseline seed the intervals cannot account for training randomness, which for Santali is about as large as the effect.

How to read the numbers

Higher is better for BLEU, chrF and chrF++; lower is better for TER. Validation loss is per token and not comparable across vocabularies, because extension runs predict fewer, harder tokens. Subsets defined by system output are diagnostic and

partly selection effects; subsets defined by the reference are cleaner. The evaluation can show whether an extension beats its baseline on the same sentences and whether the difference exceeds test-sentence noise; it cannot show transfer to other seeds or splits, or human judgement of quality (no human evaluation was done).

4.5 Tokenizer Audit

4.5.1 Vocabularies and checkpoints

SHA-256 fingerprints of the ordered piece lists identified exactly two SentencePiece vocabularies across the checkpoints inspected: a 32k English-side and a 128k Indic-side vocabulary, with consistent special tokens (BOS 0, PAD 1, EOS 2, UNK 3). The English–Indic and Indic–Indic 1B checkpoints have identical target vocabularies in identical ID order.

Candidate checkpoint	Source vocabulary	Target vocabulary
Indic to English 1B	128k	32k
English to Indic 1B	32k	128k
Indic to Indic 1B (used)	128k	128k
English to Indic distilled 200M	32k	128k
Indic to Indic distilled 320M	128k	128k

Table 8: Vocabulary sizes of the candidate IndicTrans2 checkpoints.

The Indic–Indic 1B model has 18 encoder and 18 decoder layers, hidden width 1,024, feed-forward width 8,192, 16 heads, a source vocabulary of 122,706 and a target vocabulary of 122,672 entries. Although the configuration sets embedding tying to true, the instantiated model has separate decoder input and output embeddings, so an extension must add rows to *both* matrices. The SentencePiece model has 128,000 pieces but the target vocabulary only 122,672, so a piece the segmenter picks can be missing and become `<unk>`.

4.5.2 Fertility and word splitting

Fertility is the mean number of pieces per whitespace-delimited word; Table 9 gives 2,000-sentence samples per language.

The Santali 1.4421 used in the extension runs comes from the first 2,000 non-empty training targets (38,974 words, 56,205 pieces). The Bhili baseline notebook measured its own samples (106.3 tokens per sentence and 98.4% split words at 32k; 30.9, 1.45 and 31.9% at 128k), and a second sample (31.233 tokens per sentence) is the original length used in the Bhili extension runs. The 128k vocabulary fits both

Language	Vocabulary	Tokens / sentence	Tokens / word	Words split
Santali	32k	39.544	2.029	99.574%
Santali	128k	28.102	1.442	27.239%
Bhili	32k	105.506	4.957	98.264%
Bhili	128k	30.831	1.448	32.350%

Table 9: Tokenization under the 32k and 128k vocabularies (2,000 sentences each).

languages better, but more than a quarter of Santali words and almost a third of Bhili words are still fragmented, which motivates extension.

4.5.3 Unknown tokens, round trips and digits

Santali. In the *model's own* vocabulary about 1.6% of target tokens map to unknown: 893 of 56,205 pieces (1.59%) on the 2,000-sentence sample, in both original and extended encoding, while the raw SentencePiece model produces none; on the first 100 validation sentences 54 of 2,723 tokens (1.98%) are unknown, affecting 29 sentences. The cause is the gap between 128,000 pieces and 122,672 vocabulary entries. The raw SentencePiece model round-trips 97 of 100 sentences exactly (the other three differ in whitespace), but the model-level round trip falls to 69%. Separately, 173 of the 994 test references (17.4%) contain Ol Chiki digits and no system examined ever produces one; the tokenizer represents all ten digits as single pieces, so this is a model or data issue. Removing digits from the references changes BLEU by only +0.23 to +0.28 and chrF by about +0.5, so we did not pursue it.

Bhili. The unknown-token rate is much lower: 0.061% of tokens (2 of 3,260), affecting about 1% of sentences. The Bhili baseline notebook round-tripped 93 of 100 sentences exactly (the other 7 differ only by Unicode normalisation, and NFC fixed all of them), whereas the exploration notebook reported 83 to 84 of 100 at model level on a different sample; the two should be reconciled before either is quoted. Bhili text also contains zero-width joiners, and in printed examples a rare piece (ID 106389) appears at those positions (our reading of printed pieces). The Santali and Bhili rates come from different samples and are not strictly comparable.

4.5.4 Sanity checks before any training

For both languages the trainable parameters were verified to be only the LoRA weights (3,538,944 of 1,211,635,712, 0.2921%), one optimiser step changed a tracked LoRA weight, and batch construction was checked (inputs inside the encoder vocabulary, labels inside the decoder vocabulary, -100 padding, EOS at the end). A single untuned batch gave a finite loss of about 1.74 for Santali and 9.41 for Bhili

(about 8.96 before the first update), the latter reflecting the unsupported language and proxy tag.

Chapter 5

Results

All numbers come from the saved notebook outputs. Every system was trained with the recipe of Table 6 and scored with the protocol of Chapter 4. The two languages are reported side by side in each section. Every baseline is trained exactly like the extensions of its language (Section 5.8). Two differences in control apply throughout: the Santali baseline has two seeds whereas the Bhili baseline has one run, and every Bhili run conditions on the proxy tag `mar_Deva`, so Bhili results say nothing about native support.

5.1 Main Results

Table 10 gives test scores for every system. The reference word-length ratio is 1.0: Santali systems over-generate by about 5% to 9%, Bhili systems slightly under-generate (0.980 to 0.987), and there are no empty outputs.

Santali. Changing only the seed from 42 to 43 moved baseline BLEU by +0.38 and chrF by +0.38; that is the yardstick for everything else. The 95-word system (12.31 BLEU, 45.59 chrF) sits inside that noise, while the 445- and 1,220-word systems score above both baseline seeds on chrF (46.04 to 46.17 against 45.26 and 45.64). **Bhili.** Every extension scores above the baseline on BLEU (+0.09 to +0.55) and TER (69.00 to 70.26 against 70.33). On chrF the 166-word run is 0.03 below the baseline and the other runs are 0.06 to 0.47 above it; the strongest system is the 2,729-word seed-42 run (13.40 BLEU, 44.78 chrF, TER 69.00). The 166- and 510-word runs are within a few tenths of the baseline on every metric.

5.2 Paired Bootstrap

Table 11 gives paired bootstrap differences. For Santali each extension is compared with both baseline seeds (the 1,220-word run with seed 42 only); for Bhili with the

Language	System	BLEU	chrF	TER	Len.
Santali	Baseline, seed 42	12.32	45.26	76.99	1.071
	Baseline, seed 43	12.70	45.64	77.07	1.074
	95 words, mean, s42	12.31	45.59	76.66	1.074
	445 words, mean, s42	13.15	46.15	73.98	1.054
	445 words, mean, s43	13.00	46.17	76.00	1.077
	445 words, random, s42	12.76	46.04	75.11	1.064
	1,220 words, mean, s42	13.02	46.10	77.21	1.087
Bhili	Baseline, seed 42	12.85	44.31	70.33	0.987
	166 words, mean, s42	12.94	44.28	70.26	0.987
	510 words, mean, s42	13.09	44.37	69.96	0.986
	1,173 words, mean, s42	13.27	44.75	69.09	0.980
	2,729 words, mean, s42	13.40	44.78	69.00	0.982
	2,729 words, mean, s43	13.38	44.65	69.37	0.984
	2,729 words, random, s42	13.11	44.58	69.64	0.987

Table 10: Test results (Santali 994 sentences, Bhili 990). Every baseline was trained with the same loop as the extensions of its language. Absolute scores of the two languages are not comparable.

single baseline.

Santali. Averaged over seeds, mean initialisation at 445 words gets 13.08 BLEU and 46.16 chrF against 12.51 and 45.45 for the two baselines, about +0.57 BLEU and +0.71 chrF. The chrF evidence is sturdier: all four mean-initialisation comparisons stay above zero, while BLEU intervals include zero against the stronger baseline seed. The baseline seeds differ by about +0.4 BLEU and chrF (bootstrap +0.37 BLEU [−0.12, +0.88], +0.38 chrF [−0.02, +0.77]), and the two mean-initialisation seeds by −0.15 BLEU and +0.03 chrF. The effect is real in direction and small in size. On the 300 validation sentences BLEU differences are flat (−0.23 to +0.07) and chrF differences are positive (+0.38 to +0.65) with no interval excluding zero; baseline seed 43 against seed 42 gives +0.73 chrF [+0.25, +1.22] there, so the seed spread is as large as any extension gain, and we read validation as a weak, non-contradicting check. **Bhili.** The intervals exclude zero for the 1,173-word run (BLEU and chrF; the extension wins in 98.3% and 99.9% of resamples) and for the 2,729-word seed-42 run (99.4% and 100%), and for the seed-43 run on BLEU only (chrF [−0.12, +0.78]). They include zero for the 166-word run (extension better in 67.9% of BLEU and 36.9% of chrF resamples), the 510-word run and the random-initialised run (Figure 6). This reflects resampling of a fixed test set only; with one baseline seed it is no evidence about new training runs or new data.

Language	Run	vs	ΔBLEU [95% interval]	ΔchrF [95% interval]
Santali	Mean 445, s42	baseline s42	+0.82 [+0.32, +1.41]	+0.89 [+0.44, +1.38]
	Mean 445, s42	baseline s43	+0.45 [−0.09, +1.01]	+0.50 [+0.10, +0.92]
	Mean 445, s43	baseline s42	+0.67 [+0.01, +1.38]	+0.92 [+0.38, +1.48]
	Mean 445, s43	baseline s43	+0.30 [−0.30, +0.89]	+0.53 [+0.05, +0.99]
	Random 445, s42	baseline s42	+0.43 [−0.12, +1.02]	+0.78 [+0.31, +1.26]
	Random 445, s42	baseline s43	+0.06 [−0.53, +0.65]	+0.40 [−0.04, +0.88]
	Mean 1,220, s42	baseline s42	+0.70 [+0.06, +1.35]	+0.85 [+0.38, +1.32]
Bhili	Mean 166, s42	baseline	+0.08 [−0.26, +0.42]	−0.04 [−0.30, +0.23]
	Mean 510, s42	baseline	+0.23 [−0.16, +0.63]	+0.05 [−0.26, +0.39]
	Mean 1,173, s42	baseline	+0.42 [+0.03, +0.82]	+0.44 [+0.14, +0.75]
	Mean 2,729, s42	baseline	+0.55 [+0.16, +1.00]	+0.48 [+0.14, +0.82]
	Mean 2,729, s43	baseline	+0.53 [+0.09, +0.96]	+0.35 [−0.12, +0.78]
	Random 2,729, s42	baseline	+0.25 [−0.18, +0.68]	+0.27 [−0.09, +0.64]

Table 11: Paired bootstrap differences on the test sets (extension minus baseline, 1,000 resamples). All Bhili intervals come from one comparison notebook that scores the saved test predictions of every run against the baseline.

5.3 Effect of the Number of Added Words

Figure 7 plots the chrF gain over the baseline against list size. The Santali curve rises to about 0.9 chrF at 445 words and stays there; the Bhili curve is flat at zero up to 510 words and then rises to about 0.45 chrF at 1,173 and 2,729 words. Total gains are a few tenths of a chrF point in both.

Santali. Test chrF is 45.26 and 45.64 for the baselines, 45.59 for 95 words, 46.15 for 445 and 46.10 for 1,220: the 95-word list (1.89% token saving) is too small to matter, and the 1,220-word list saves more tokens (7.36%) but scores no better than 445. **Bhili.** The 166- and 510-word runs cannot be told from the baseline (chrF −0.04 and +0.05, BLEU +0.08 and +0.23, all intervals include zero). A gain appears at 1,173 words (+0.42 BLEU, +0.44 chrF) and the 2,729-word run is not meaningfully larger: the two differ by only 0.13 BLEU and 0.03 chrF, far inside the bootstrap interval widths (about ±0.4 BLEU). A detectable gain therefore starts later than in Santali (1,173 against 445 words) although equal thresholds give lists of similar size (1,173 and 1,220 words at threshold 10), and it is smaller than the Santali gain over the same-seed baseline (+0.82 BLEU, +0.89 chrF at 445 words).

Table 12 shows how much of the added vocabulary is generated on the test set. In both languages the share of added words ever used falls steadily as the list grows, even though larger lists remove more pieces (up to 7.36% of validation tokens for Santali and 11.26% on training text for Bhili). Extra tokens covering rarer words cost more output classes and more rows to learn from fewer examples, without a clear return.

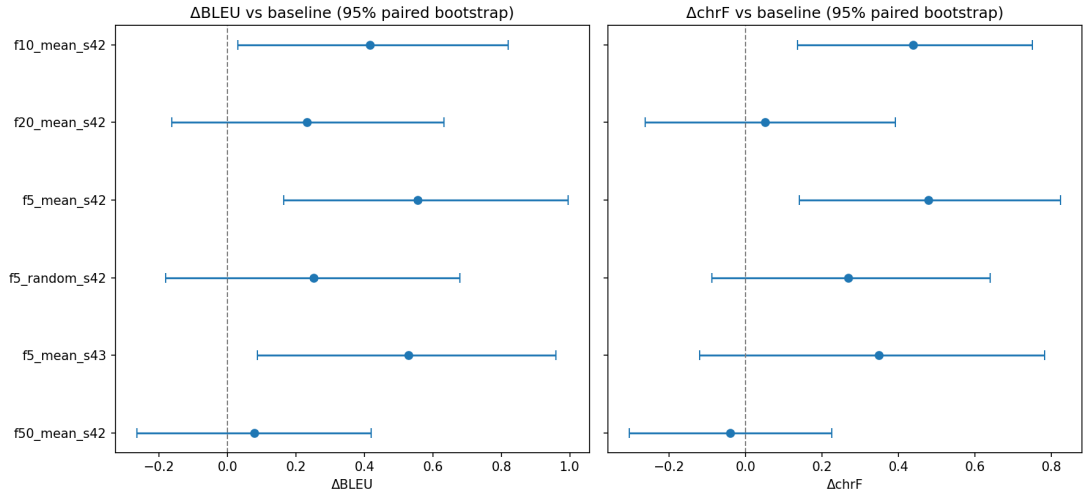


Figure 6: Bhili: ΔBLEU and ΔchrF of each extension run over the baseline with 95% paired bootstrap intervals. Labels give the minimum frequency, initialisation and seed: `f50` is 166 added words, `f20` 510, `f10` 1,173 and `f5` 2,729.

5.4 Mean Versus Random Initialisation

Santali. At 445 words mean initialisation scores 0.39 BLEU [−0.02, +0.80] and 0.10 chrF [−0.26, +0.47] above random at seed 42, and 0.24 BLEU and 0.13 chrF above it with the second mean seed; all four intervals include zero. The mean model learns faster early (epoch-1 validation loss 2.739 against 2.825) and uses an added token in more test sentences (507 against 452), but by epoch 10 the loss gap is only about 0.01 (2.589 against 2.598). **Bhili.** At 2,729 words random scores 13.11 BLEU and 44.58 chrF against 13.38 to 13.40 and 44.65 to 44.78 for the two mean runs (mean ahead by 0.27 to 0.29 BLEU and 0.07 to 0.20 chrF), has a worse validation loss (4.0894 against 4.0510; epoch 1: 4.79 against 4.63) and uses fewer distinct added words (377 against 449 and 453). Against the baseline the random run gains +0.25 BLEU [−0.18, +0.68] and +0.27 chrF [−0.09, +0.64], intervals that include zero, whereas the seed-42 mean run's exclude it. The direction is the same in every comparison, but gaps are comparable to the spread between identical mean runs, there is a single random run per language, and no direct bootstrap between initialisations was run for Bhili. In both languages mean initialisation is a safe default and probably slightly better, but not a proven win; starting values matter little once training is long enough.

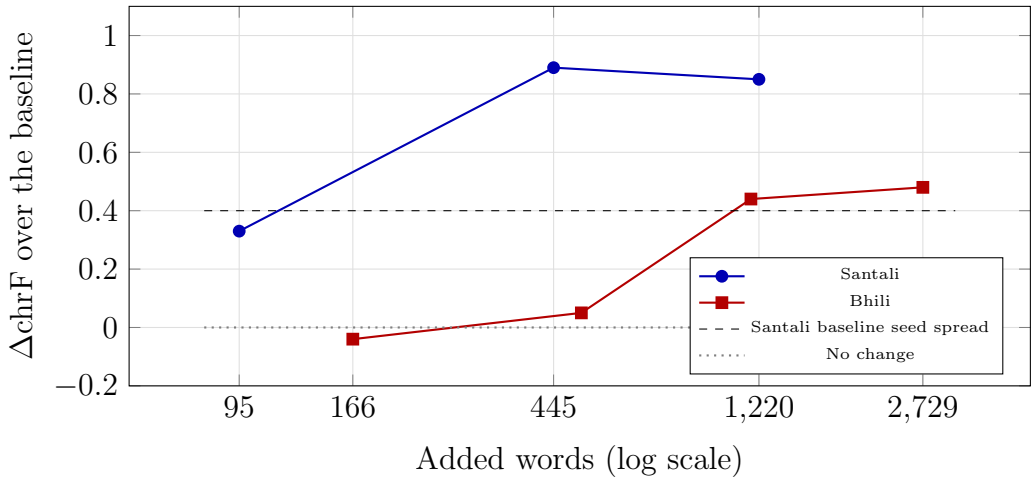


Figure 7: Test chrF gain over the baseline against list size. The dashed line is the Santali seed-to-seed spread of the unextended baseline (about 0.4 chrF). Both languages are compared with a matched baseline (Santali against seed 42); the Bhili baseline has one run.

5.5 Seed Variation

Santali. The two mean-initialised 445-word seeds differ by −0.15 BLEU and +0.02 chrF, while the baseline's own seed spread is about 0.4 BLEU and chrF, as large as the extension effect. **Bhili.** A change of seed at 2,729 words moves the extension by −0.02 BLEU and −0.13 chrF, so the extension is stable to about 0.1 BLEU and 0.13 chrF. This does not tell us the noise of the Bhili *baseline*, which has one run: if it is like Santali's (about 0.4), the matched gains of 0.42 to 0.55 BLEU at 1,173 and 2,729 words are about that size, and the smaller gains at 166 and 510 words are well inside it.

5.6 Where the Gain Comes From

If the vocabulary caused the gain, it should be larger on sentences that contain added words. Table 13 splits test sentences by whether the system's own *output* contains an added token. A caveat is that this split is partly a selection effect, so we also split by whether the *reference* contains an added word, the cleaner test.

Santali. The gain sits in sentences that use added tokens and is close to zero elsewhere. Split by reference, the 445-word mean runs gain about +1.1 BLEU and +1.1 to +1.3 chrF where the reference contains an added word (576 sentences) against −0.11 to +0.41 BLEU and +0.21 to +0.56 chrF where it does not (418 sentences). The random run shows no such contrast (+0.46 BLEU and +0.91 chrF

Language	Run	Added	Distinct used	Share used	Sent. with added word
Santali	95 words	95	74	78%	288
	445, mean, s42	445	262	59%	507
	445, mean, s43	445	265	60%	507
	445, random	445	246	55%	452
	1,220 words	1,220	511	42%	620
Bhili	166 words	166	103	62%	675
	510 words	510	210	41%	739
	1,173 words	1,173	301	26%	769
	2,729, mean, s42	2,729	449	16%	791
	2,729, mean, s43	2,729	453	17%	792
	2,729, random	2,729	377	14%	770

Table 12: Use of added words in test outputs (Santali 994 sentences, Bhili 990). Share used is distinct words generated over words added. Mean initialisation puts more added words into use than random initialisation in both languages.

against +0.41 and +0.55), and the 95-word run none either. **Bhili.** Split by output, the chrF gain sits in sentences that use an added word for the 510-, 2,729 and random runs (2,729 s42: +0.55 against +0.03; s43: +0.63 against −1.43; random: +0.45 against −0.75); the 1,173-word run is the exception (+0.43 against +0.48). The BLEU gain is not concentrated in the same way (for example +0.72 where the 2,729-word s42 output uses no added word). The groups without an added word are small (183 to 302 sentences) and noisy. Split by reference, the cleaner test (Table 14), the chrF gain is larger where the reference contains an added word for the 1,173-word run (+0.51 against −0.06), the 2,729-word s42 run (+0.54 against −0.21) and the random run (+0.35 against −0.60), and similar in both groups for seed 43 (+0.36 against +0.28), while BLEU gains are similar in both groups for the 2,729-word mean runs (+0.55 against +0.54; +0.52 against +0.58) and larger where the reference has an added word only for the 1,173-word run (+0.49 against −0.10). The chrF improvement therefore looks word-related, as in Santali, but BLEU shows it in one run only, and the no-added-word groups (142 to 353 sentences) are small, so this is suggestive, not conclusive.

Word level (Santali). For each of the 445 added words (1,091 occurrences in the test references) we measured how often it is produced correctly, against a control of 775 multi-piece words with training frequency 10 to 19 that were not added (499 reference occurrences; Table 15). This analysis was not run for Bhili.

Against the baseline average, recall of the added words changes by +1.88 [−0.22, +4.04], +1.26 [−0.74, +3.33] and +2.81 [+0.79, +4.85] points while the control changes by −0.08, −0.27 and −0.69, and precision of the added words rises by

Language	Run	Output uses added token n	ΔBLEU / ΔchrF	Output uses none n	ΔBLEU / ΔchrF
Santali	95 words	288	+0.19 / +0.84	706	−0.11 / +0.09
	445, s42	507	+1.51 / +1.65	487	−0.01 / −0.03
	445, s43	507	+1.42 / +1.63	487	−0.20 / +0.06
	445, random	452	+0.94 / +1.57	542	−0.07 / +0.01
	1,220 words	620	+1.35 / +1.51	374	−0.54 / −0.43
Bhili	166 words	688	+0.10 / −0.01	302	+0.10 / −0.10
	510 words	750	+0.26 / +0.16	240	+0.13 / −0.40
	1,173 words	777	+0.43 / +0.43	213	+0.58 / +0.48
	2,729, s42	807	+0.60 / +0.55	183	+0.72 / +0.03
	2,729, s43	806	+0.66 / +0.63	184	−0.31 / −1.43
	2,729, random	787	+0.37 / +0.45	203	−0.36 / −0.75

Table 13: Change against the baseline on test sentences, split by the system's own output (Santali against seed 42). For Bhili the split is estimated by matching added words in the output text, because the comparison notebook had no generated-ID files, so its counts differ slightly from Table 12.

Language	Run	Reference contains added word n	ΔBLEU / ΔchrF	Reference contains none n	ΔBLEU / ΔchrF
Bhili	166 words	637	+0.04 / −0.04	353	+0.39 / +0.00
	510 words	744	+0.22 / +0.09	246	+0.34 / −0.06
	1,173 words	797	+0.49 / +0.51	193	−0.10 / −0.06
	2,729, s42	848	+0.55 / +0.54	142	+0.54 / −0.21
	2,729, s43	848	+0.52 / +0.36	142	+0.58 / +0.28
	2,729, random	848	+0.27 / +0.35	142	+0.08 / −0.60

Table 14: Bhili: change against the baseline on test sentences, split by whether the reference contains an added word.

about 3.6 to 5.8 points against −0.3 to +0.7 for the control (precision was not bootstrapped). The added tokens help mostly by being *produced correctly* more than by being found more often. A looser control of 1,328 frequent words that were not added gained about +1.1 to +1.3 points of recall, so a small general improvement also exists. In Santali, then, the gain is at best partly specific to the added words, and the control set is small and its intervals wide, so we do not over-read it; the Bhili sentence-level splits point to a word-related chrF gain, but no word-level control was run for Bhili.

5.7 Training Behaviour

Every run completed all ten epochs and the best epoch was the last or last-but-one in every case. Table 16 and Figure 8 give validation loss per epoch for selected systems (per token, not comparable across vocabulary sizes). Curves converge smoothly in

System	Added rec.	Added prec.	Ctrl rec.	Ctrl prec.
Baseline, seed 42	41.43	26.56	32.26	31.45
Baseline, seed 43	41.70	25.46	33.27	32.17
Mean init, seed 42	43.45	29.57	32.67	31.90
Mean init, seed 43	42.80	30.48	32.46	32.46
Random init, seed 42	44.36	31.76	32.06	31.56

Table 15: Word-level recall (rec.) and precision (prec.) in % for the 445 added words and the matched control (Ctrl), Santali.

both languages, and the best epoch of every baseline is its last. In both languages every extension curve sits above its baseline curve from the first epoch on, as expected when the extension predicts fewer, harder tokens (Section 5.8): at epoch 10 by 0.12 to 0.19 for Santali (2.5885 and 2.6556 against 2.4664) and by 0.11 to 0.17 for Bhili (4.0366 to 4.0894 against 3.9231).

	Santali			Bhili			
Epoch	Base s42	445 s42	1,220	Baseline	166	2,729 s42	2,729 rand.
1	2.5847	2.7390	2.8372	4.5063	4.5962	4.6309	4.7891
2	2.5399	2.6723	2.7525	4.2837	4.3945	4.3991	4.4865
3	2.5141	2.6461	2.7191	4.1505	4.2665	4.2670	4.3329
4	2.4951	2.6234	2.6936	4.0782	4.1950	4.1960	4.2467
5	2.4812	2.6143	2.6793	4.0172	4.1343	4.1341	4.1810
6	2.4764	2.6038	2.6702	3.9777	4.0925	4.0987	4.1398
7	2.4739	2.5993	2.6642	3.9539	4.0694	4.0766	4.1160
8	2.4678	2.5914	2.6575	3.9409	4.0545	4.0590	4.1046
9	2.4662	2.5886	2.6553	3.9310	4.0450	4.0563	4.0944
10	2.4664	2.5885	2.6556	3.9231	4.0366	4.0510	4.0894

Table 16: Validation loss per epoch (per token; not comparable across vocabulary sizes).

5.8 Baseline Comparability

Santali. Both baselines use the extension loop (fp16 autocast, gradient scaler, same schedule and checkpoint logic), so extension and baseline differ only in the vocabulary, and the comparison is matched.

Bhili. The Bhili baseline notebook is the extension notebook with the vocabulary step removed: the same split sizes (asserted as 17,817 / 990 / 990), tokenizer and IndicProcessor prompts, LoRA configuration (3,538,944 trainable parameters, asserted), optimiser, schedule (11,140 steps, 334 warmup), fp16 autocast with a gradient scaler and skipping of non-finite updates, checkpointing and evaluation settings (5 beams, 256 new tokens); its decoder check passes (0 mismatches on 600 sentences). The Bhili comparison is therefore matched, like the Santali one. Two limits remain. The baseline has one seed, so its seed noise is unknown, whereas the

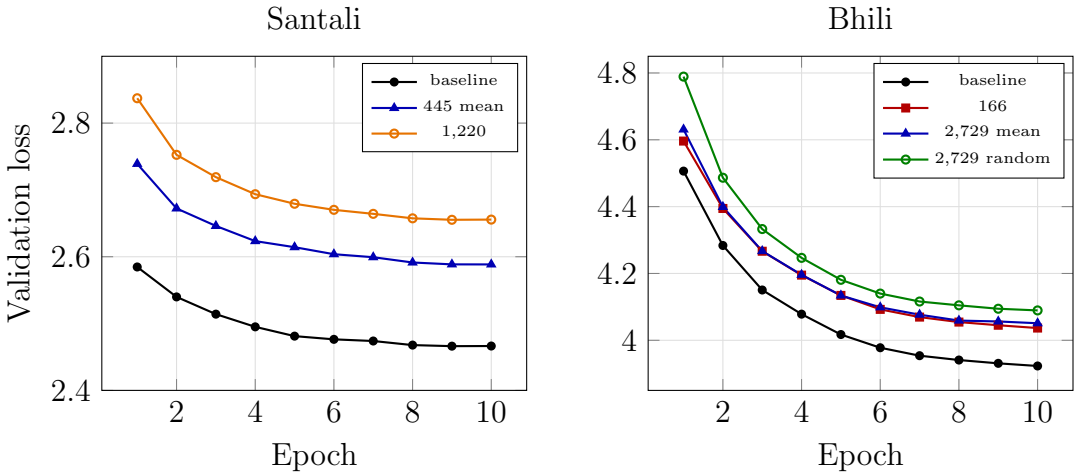


Figure 8: Validation loss per epoch for Santali (left) and Bhili (right) (Table 16). Loss is per token, so curves for different vocabulary sizes are not directly comparable.

two Santali baseline seeds differ by about 0.4 BLEU and chrF, about the size of the Bhili gains. And validation loss, which is per token, is higher for every extension than for the baseline in both languages by similar margins (Section 5.7), as expected for a larger vocabulary that predicts fewer, harder tokens; it is not evidence for or against a gain. A second Bhili baseline seed is the experiment that would settle how large the Bhili gain is relative to training noise (Section 7.3).

5.9 External Evaluation on IN22-Gen

Santali. The trained systems were evaluated without further training on IN22-Gen, the independent multi-domain benchmark released with IndicTrans2 (1,024 English–Santali pairs), with the same prompts, decoder and sacreBLEU settings, plus chrF++. One benchmark sentence overlaps the training English text and none of the Santali references do; scores on the 1,023 non-overlapping sentences are within 0.02 chrF of the full set. Coverage matters because an extension trained on one domain helps only where its words occur: the 445-word list covers 3.30% of reference words on IN22-Gen against 5.62% on the in-domain test split (token reduction 2.50% against 4.92%), and the 1,220-word list 6.33% against 8.19%. Unknown tokens make up 2.44% of the IN22-Gen references under the original tokenizer. **Bhili.** No independent external benchmark exists in this work; the Bhili metrics describe only the local held-out split.

Three things stand out. All three 445-word systems beat both baseline seeds on chrF++ with intervals excluding zero (+0.56 to +0.85), so the direction survives a change of domain despite lower coverage. BLEU gains are smaller and mostly not

System	BLEU	chrF	chrF++	TER	Len. ratio
Pretrained, no adaptation	4.82	34.72	31.36	104.33	1.210
Baseline, seed 42	5.88	37.53	33.75	96.53	1.144
Baseline, seed 43	5.92	37.51	33.77	96.82	1.143
445 words, mean, seed 42	6.06	38.18	34.43	94.98	1.139
445 words, mean, seed 43	6.32	38.33	34.59	93.75	1.128
445 words, random, seed 42	5.98	38.12	34.34	96.73	1.158

Table 17: Santali results on IN22-Gen (1,024 sentences). The 95- and 1,220-word models were not evaluated here.

Run	vs	ΔBLEU	ΔchrF++
Mean 445, s42	baseline s42	+0.18 [−0.19, +0.54]	+0.69 [+0.25, +1.10]
Mean 445, s42	baseline s43	+0.13 [−0.26, +0.51]	+0.66 [+0.23, +1.10]
Mean 445, s43	baseline s42	+0.44 [+0.15, +0.73]	+0.85 [+0.45, +1.28]
Mean 445, s43	baseline s43	+0.40 [+0.09, +0.70]	+0.82 [+0.43, +1.27]
Random 445, s42	baseline s42	+0.10 [−0.25, +0.42]	+0.59 [+0.17, +1.00]
Random 445, s42	baseline s43	+0.06 [−0.32, +0.46]	+0.56 [+0.08, +1.04]
Baseline s43	baseline s42	+0.04 [−0.20, +0.29]	+0.03 [−0.37, +0.43]
Baseline s42	pretrained	+1.06 [+0.62, +1.49]	+2.38 [+1.67, +3.03]

Table 18: Paired bootstrap on IN22-Gen, Santali (95% intervals over benchmark sentences).

significant, and the two baseline seeds differ by only 0.04 BLEU and 0.03 chrF++ here, so seed noise is small relative to the gain on this benchmark. And fine-tuning matters much more than vocabulary: LoRA lifts chrF++ over the unadapted model by about 2.4 points, while extension adds roughly 0.7. On the 532 of 1,024 sentences whose reference contains an added word, chrF++ gains range from about +0.55 to +1.25, and from +0.29 to +0.74 where it does not, so the general part of the gain is relatively larger out of domain. The 445-word systems emit an added token in 312 to 377 sentences. For orientation, a participant study reports 7.3 BLEU and 40.3 chrF++ on IN22-Gen with extra public data [16]; our best (6.32 BLEU, 34.59 chrF++) is lower, as expected for a single direction, only the shared-task split and a LoRA adapter, and different scoring details, and is cited as a rough sanity range only.

Chapter 6

Discussion

6.1 Santali: the Controlled Result

Extending the target vocabulary with a few hundred frequent whole Santali words gives about +0.6 BLEU and +0.7 chrF in-domain and +0.56 to +0.85 chrF++ on IN22-Gen. The improvement is larger in sentences that use the added words and shows up partly as more accurate production of those words, but frequent words that were *not* added improved too, so part of the benefit is general. Too few words (95) do nothing measurable and too many (1,220) do no more than 445; compositional initialisation is at best slightly better than random. The pattern echoes Singh et al. [18]: reduced fragmentation alone does not guarantee better translation (1,220 words save 7.36% of tokens and score no better than 445, which save 4.73%). What the evidence does not support is equally important: the effect is the size of the baseline's seed-to-seed variation, only the 445-word setting has two seeds, and the validation BLEU gain does not appear. We describe a small effect that points the same way in every comparison, not a large improvement.

6.2 Bhili: a Replication With a Matched Baseline

Against a baseline trained with the identical loop, the Bhili study reproduces part of the Santali result. A short list does nothing (no measurable effect at 166 or 510 words), most added words are never generated (16% of the 2,729 are used), mean initialisation is not clearly better than random, and the extension is stable across a second seed (about 0.1 BLEU and chrF). It differs in two ways. A gain needs a longer list: it becomes detectable at 1,173 words (+0.42 BLEU, +0.44 chrF) and is no larger at 2,729 (+0.55, +0.48), against 445 words in Santali. And it is smaller than the Santali gain over the same-seed baseline (+0.82 BLEU, +0.89 chrF), roughly the size of the Santali baseline's seed spread of 0.4. The chrF gain sits mostly in

sentences with an added word, the pattern a word-specific explanation predicts, but BLEU shows it in one run only and no word-level control was run. Two controls that Santali has are missing: a second baseline seed and an external benchmark. The careful statement is that Bhili extensions with 1,173 or more words score about 0.4 to 0.55 BLEU and 0.35 to 0.48 chrF above a matched, single-seed baseline: a small effect of consistent sign whose size cannot yet be separated from seed noise.

6.3 The Methodological Lesson

The effect we measured is small, about the size of the Santali baseline's own seed-to-seed spread, so it is visible only because every system used the same procedure as its baseline, the Santali baseline was run at two seeds and differences were paired and bootstrapped. Decoding spacing alone moved BLEU by 0.6, and the Bhili gain, 0.4 to 0.55 BLEU over a single-seed baseline, is of a size that a second baseline seed could confirm or erase. Anyone comparing vocabulary-adaptation methods should match the training procedure, run a second seed, look at where in the data the gain appears and report null runs.

6.4 Claims the Evidence Supports and Does Not Support

Claim	Santali	Bhili	Suggested wording
LoRA fine-tuning beats the pretrained model	Supported (IN22-Gen: +1.1 BLEU, +2.4 chrF++)	Not tested	"LoRA adds about 1.1 BLEU and 2.4 chrF++ on IN22-Gen (Santali)"
Vocabulary extension improves chrF	Supported with caveats (445 to 1,220 words)	Supported with caveats (+0.44 to +0.48 chrF at 1,173 and 2,729 words; single baseline seed)	"A modest, repeatable chrF gain in Santali; a smaller gain in Bhili with 1,173 or more words"
A very short list does nothing	Supported (95 words, single seed)	Supported (166 and 510 words, single seed)	"No measurable effect at 95 Santali or 166 to 510 Bhili words"
The gain generalises to an external benchmark	Supported on chrF++, weak on BLEU	No benchmark	"Carries over on chrF++ (Santali)"
The gain comes from the added words	Not supported (control words improve too)	Tentative on chrF (estimated split); BLEU mixed	"Part of the benefit appears general in Santali"
Mean initialisation beats random	Unresolved	Unresolved	"No reliable difference"
The gain flattens with list size	Supported (about 445 words)	Tentative (nothing below 1,173 words, nothing more at 2,729)	"Appears to flatten"
Training and decoding details can create or hide gains	Supported (decoding spacing alone: +0.6 BLEU)	Not tested	Lead with this as a methodological point
IndicTrans2 supports Bhili	–	Not supported (proxy tag only)	"Conditioned on a proxy tag"

Table 19: Claims, their status for each language, and suggested wording.

Chapter 7

Conclusion and Future Work

7.1 Conclusion

This thesis asked whether adding whole-word tokens for frequent target words to IndicTrans2's decoder vocabulary improves translation into two Indian tribal languages under LoRA fine-tuning.

For Santali (English→Santali, officially supported) the answer under matched training and evaluation is a small but consistent yes at several hundred words: roughly +0.6 BLEU and +0.7 chrF on the held-out split and +0.56 to +0.85 chrF++ on IN22-Gen. The gain is larger in sentences containing the added words, though control words improved too, so it is not purely word-specific. 95 words are too few, 1,220 add nothing beyond 445, and compositional initialisation is not clearly better than random. The gain is comparable in size to seed variation: a modest effect of consistent direction, not a breakthrough.

For Bhili (Hindi→Bhili, proxy tag `mar_Deva`), against a baseline trained with the identical loop, the effect is smaller and needs longer lists. Lists of 166 and 510 words are indistinguishable from the baseline (+0.08 and +0.23 BLEU, −0.04 and +0.05 chrF, all intervals include zero). Lists of 1,173 and 2,729 words score +0.42 and +0.55 BLEU and +0.44 and +0.48 chrF above it with intervals that exclude zero; the second 2,729-word seed gives +0.53 BLEU (excludes zero) and +0.35 chrF (includes zero), and random initialisation +0.25 BLEU and +0.27 chrF (both include zero). The extension is stable across the two seeds, most of the 2,729 added words are never generated, and the chrF gain sits mostly in sentences with an added word. The baseline has one seed, so the Bhili gain, about the size of the Santali baseline's seed spread, is a small effect of consistent sign and not a measured effect of fixed size. Nothing here shows that IndicTrans2 supports Bhili.

The common lesson is methodological: matching the training procedure, repeating seeds and looking at where a gain appears were essential to reading both languages correctly, and in both the effect is small enough that the baseline's own

seed noise is the main uncertainty.

7.2 Limitations

Both languages

- **Test-set reuse and bootstrap scope.** Several configurations were scored on the same test set, and the intervals cover test-sentence sampling, not training randomness.
- **Internal test set and possible leakage.** All numbers come from a local held-out split, not the official test; repeated sources (Santali 2,412 rows, Bhili 472) may cross splits and were not measured. No human evaluation was done.
- **Subset and control design.** The output-based split is partly a selection effect (for Bhili also estimated from output text), and the Santali control set is small, so word-specificity is not settled.
- **Reference noise** (including an apparent misaligned Bhili pair) was not audited.
- **Capacity versus tokenization.** A control with added rows the labels never use was not run, so "extra trainable rows" and "shorter sequences" are not separated.
- **Metrics.** sacreBLEU defaults were used, not official IndicNLP-tokenised scoring; exact-match word metrics understate quality for morphologically rich languages.

Santali

- **Few runs.** Two baseline seeds, two seeds for 445 mean-initialised words, one run each for 95 words, random initialisation and 1,220 words; the baseline's seed spread is comparable to the in-domain effect.
- **Validation does not confirm the BLEU gain**; the 300-sentence sample is small and was used to pick checkpoints.
- **IN22-Gen is incomplete.** The 95- and 1,220-word systems were not evaluated; one English source overlaps training; scoring is not IndicNLP-tokenised.

- **Missing pieces and resumed runs.** 3 of 1,220 added words contain a piece missing from the vocabulary, giving a small unrelated gain; the random-initialisation run was resumed from a checkpoint. The unknown-token diagnostics notebook was saved without outputs and the Ol Chiki digit check covered two systems only.

Bhili

- **One baseline run, limited seeds.** The baseline has seed 42 only, so Bhili seed noise is unknown, and with Santali's baseline spread of about 0.4 BLEU and chrF the matched Bhili gains of 0.4 to 0.55 BLEU cannot be separated from it. This is the most important limitation of the Bhili result. Extension seeds 42 and 43 exist only at 2,729 words with mean initialisation; the 166-, 510- and 1,173-word and random-initialisation runs are single seed-42 runs.

- **Estimated subsets.** The comparison notebook had no generated-ID files, so the output-based split uses text matching and its counts differ slightly from the usage table; the word-level control analysis was not run for Bhili.

- **Parity check failures.** The 510-, 1,173- and 2,729-word runs and the seed-43 run failed their own decoder parity check (2 of 600 whitespace differences) but trained anyway; the effect was not measured.

- **Split sizes and round trips.** The notebooks use 17,817 / 990 against 17,821 / 991 in preprocessing, and the two round-trip counts (83–84 and 93 of 100) were not reconciled.

- **Proxy tag, no validation scores, no external benchmark.** Every result depends on `mar_Deva`; Bhili validation BLEU and chrF were not reported.

7.3 Future Work

In priority order:

1. **More Bhili baseline seeds.** Train the baseline with seed 43 (and more) in the same loop and compare the extensions with each; this decides whether the 0.4 to 0.55 BLEU Bhili gain exceeds training noise.

2. **Fix integrity issues.** Fix the decoder spacing rule around parentheses so parity passes before training, resolve the Bhili split-size difference and document the dataset version, and reconcile the round-trip counts.

3. **More seeds.** Seed 43 for the Bhili 510- and 1,173-word runs, a second Bhili random run with a direct mean-versus-random bootstrap, and extra seeds with seed-matched baselines for the Santali 95-, 1,220-word and random runs.

4. **Complete the curves and controls.** Add a 3,020-word and a 21-word Santali run and a 60-word Bhili run; repeat the added-versus-control word analysis for Bhili with larger matched-frequency controls, using generated token IDs instead of text matching.

5. **Understand the digits.** Find out why no system produces an Ol Chiki digit (17% of references contain them); a fix may be worth more than any vocabulary change.

6. **External evaluation, leakage and noise.** Evaluate the 95- and 1,220-word Santali systems on IN22-Gen, look for an external Bhili benchmark, rescore on a source-disjoint subset, and audit reference noise in both training sets.

7. **Broader scope.** Mundari and Gondi, the reverse direction, the distilled 200M and 320M checkpoints, subword-level extensions (stems or affixes), morphology-aware metrics, and human evaluation by Santali and Bhili speakers.

References

[1] Ahia, O., Kumar, S., Gonen, H., Kasai, J., Mortensen, D. R., Smith, N. A., and Tsvetkov, Y. Do all languages cost the same? tokenization in the era of commercial language models. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing* (2023), pp. 9904–9923.

[2] Dodge, J., Ilharco, G., Schwartz, R., Farhadi, A., Hajishirzi, H., and Smith, N. Fine-tuning pretrained language models: Weight initializations, data orders, and early stopping. *arXiv preprint arXiv:2002.06305* (2020).

[3] Gala, J., Chitale, P. A., Raghavan, A. K., Gumma, V., Doddapaneni, S., Kumar, A., Nawale, J., Sujatha, A., Puduppully, R., Raghavan, V., Kumar, P., Khapra, M. M., Dabre, R., and Kunchukuttan, A. IndicTrans2: Towards high-quality and accessible machine translation models for all 22 scheduled indian languages. *Transactions on Machine Learning Research* (2023).

[4] Hewitt, J. Initializing new word embeddings for pretrained language models. Blog post, `https://nlp.stanford.edu/~johnhew/vocab-expansion.html`, 2021.

[5] Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., and Chen, W. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations (ICLR)* (2022).

[6] Koehn, P. Statistical significance tests for machine translation evaluation. In *Proceedings of the 2004 Conference on Empirical Methods in Natural Language Processing* (2004), pp. 388–395.

[7] Kudo, T., and Richardson, J. SentencePiece: A simple and language independent subword tokenizer and detokenizer for neural text processing. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing: System Demonstrations* (2018), pp. 66–71.

[8] NLLB Team, Costa-jussà, M. R., Cross, J., et al. No language left behind: Scaling human-centered machine translation. *arXiv preprint arXiv:2207.04672* (2022).

[9] Office of the Registrar General and Census Commissioner, India. Census of india 2011: Abstract of speakers' strength of languages and mother tongues, 2011.

[10] Papineni, K., Roukos, S., Ward, T., and Zhu, W.-J. BLEU: a method for automatic evaluation of machine translation. In *Proceedings of the 40th Annual Meeting of the Association for Computational Linguistics* (2002), pp. 311–318.

[11] Petrov, A., La Malfa, E., Torr, P. H. S., and Bibi, A. Language model tokenizers introduce unfairness between languages. In *Advances in Neural Information Processing Systems* (2023), vol. 36.

[12] Popović, M. chrF: character n-gram F-score for automatic MT evaluation. In *Proceedings of the Tenth Workshop on Statistical Machine Translation* (2015), pp. 392–395.

[13] Post, M. A call for clarity in reporting BLEU scores. In *Proceedings of the Third Conference on Machine Translation: Research Papers* (2018), pp. 186–191.

[14] Raj, S. Challenge track: LoRAs in all directions – directional adapters and noisy-channel reranking for Indic MT. In *Proceedings of the 1st Workshop on Multimodal Models for Low-Resource Contexts and Social Impact (MMLoSo 2025)* (Mumbai, India, Dec. 2025), Association for Computational Linguistics, pp. 101–105.

[15] Rust, P., Pfeiffer, J., Vulić, I., Ruder, S., and Gurevych, I. How good is your tokenizer? on the monolingual performance of multilingual language models. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing* (2021), pp. 3118–3135.

[16] Singh, K. B., Ekbal, A., and Pakray, P. Evaluating IndicTrans2 and ByT5 for English–Santali machine translation using the Ol Chiki script. In *Proceedings of the 1st Workshop on Multimodal Models for Low-Resource Contexts and Social Impact (MMLoSo 2025)* (Mumbai, India, Dec. 2025), Association for Computational Linguistics, pp. 95–100.

[17] Singh, P., Chatterjee, S., Cheema, G. S., Bedi, A. S., Chakraborty, T., Kumar, S., and Shukla, A. Findings of the MMLoSo 2025 shared task on machine translation into tribal languages. In *Proceedings of the 1st Workshop on Multimodal Models for Low-Resource Contexts and Social Impact (MMLoSo 2025)* (Mumbai, India, Dec. 2025), Association for Computational Linguistics, pp. 121–129.

[18] Singh, P., Chauhan, S., Agnihotri, N., and Kumar, S. Script is not preprocessing: Tokenizer-mediated transfer in endangered language machine translation. In *Proceedings of the 2026 Conference on Empirical Methods in Natural Language Processing* (Oct. 2026), Association for Computational Linguistics, pp. 39110–39150.

[19] Wang, H., Yu, D., Sun, K., Chen, J., and Yu, D. Improving pre-trained multilingual models with vocabulary expansion. In *Proceedings of the 23rd Conference on Computational Natural Language Learning (CoNLL)* (2019), pp. 95–105.